COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 15, 2026	**DATE SUBMITTED:**	September 15, 2026
NAME:	Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 SORTING ALGORITHMS PT2

1. PROCEDURE OUTPUT

What procedures did you perform?

Code + Console Screenshot

```
20  int main(){
21
22      srand(time(0));
23
24      const int SIZE = 100;
25
26      int original[SIZE];
27      int shellArray[SIZE];
28      int mergeArray[SIZE];
29      int quickArray[SIZE];
30
31
32      // To Generate 100 random values
33      for (int i = 0; i < SIZE; i++){
34          original[i] = rand() % 1000 + 1;
35      }
```

```
44
45      cout << "ARRAY OF VALUES FOR SORT ALGORITHM TESTING" << endl;
46      cout << "Original 100 Random Values:" << endl;
47      displayArray(original, SIZE);
```

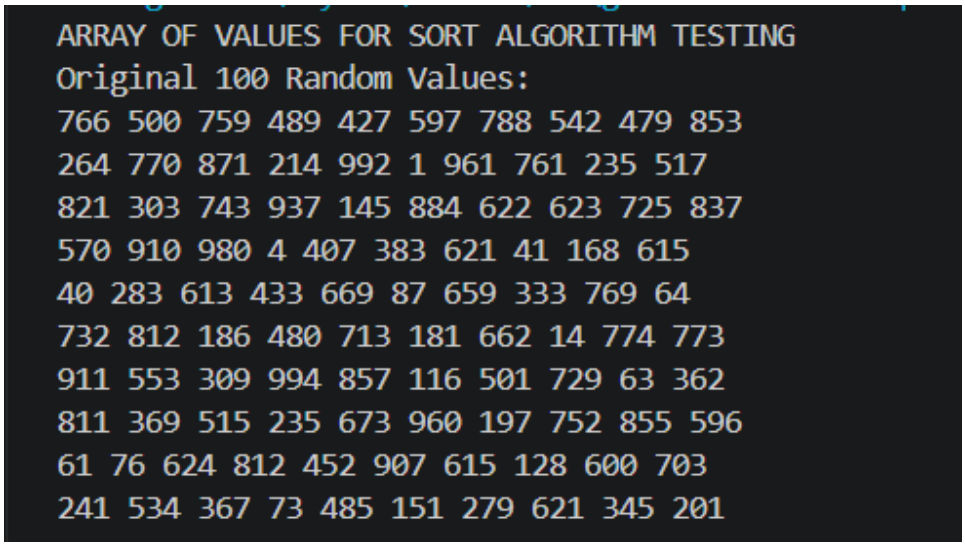
	
Observations	The program successfully generated 100 random values that were initially unsorted. The same set of values was then copied and used as the dataset for Shell Sort, Merge Sort, and Quick Sort.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	sorting.h × main.cpp ×
	sorting.h > ...
	1 #ifndef SORTING_H
	2 #define SORTING_H
	3
	4 // Shell Sort
	5 void shellSort(int arr[], int size){
	6 for (int gap = size / 2; gap > 0; gap /= 2){
	7 for (int i = gap; i < size; i++){
	8 int temp = arr[i];
	9 int j = i;
	10 while (j >= gap && arr[j - gap] > temp){
	11 arr[j] = arr[j - gap];
	12 j -= gap;
	13 }
	14 arr[j] = temp;
	15 }
	16 }
	17 }

```
48
49      cout << "\nSHELL SORT TECHNIQUE" << endl;
50      shellSort(shellArray, SIZE);
51      cout << "Sorted Array using Shell Sort:" << endl;
52      displayArray(shellArray, SIZE);
```

```
SHELL SORT TECHNIQUE
Sorted Array using Shell Sort:
1 4 14 40 41 61 63 64 73 76
87 116 128 145 151 168 181 186 197 201
214 235 235 241 264 279 283 303 309 333
345 362 367 369 383 407 427 433 452 479
480 485 489 500 501 515 517 534 542 553
570 596 597 600 613 615 615 621 621 622
623 624 659 662 669 673 703 713 725 729
732 743 752 759 761 766 769 770 773 774
788 811 812 812 821 837 853 855 857 871
884 907 910 911 937 960 961 980 992 994
```

Observations	Shell Sort successfully arranged the 100 random values in ascending order. The algorithm used different gap intervals to sort elements that were separated from each other before completing the sorting process.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
21  void merge(int arr[], int left, int middle, int right){
22      int n1 = middle - left + 1;
23      int n2 = right - middle;
24      int leftArr[n1];
25      int rightArr[n2];
26      for (int i = 0; i < n1; i++){
27          leftArr[i] = arr[left + i];
28      }
29      for (int j = 0; j < n2; j++){
30          rightArr[j] = arr[middle + 1 + j];
31      }
32      int i = 0;
33      int j = 0;
34      int k = left;
35
36      while (i < n1 && j < n2){
37          if (leftArr[i] <= rightArr[j]){
38              arr[k] = leftArr[i];
39              i++;
40          }else{
41              arr[k] = rightArr[j];
42              j++;
43          }
44
45          k++;
46      }
```

```
48          while (i < n1){
49              arr[k] = leftArr[i];
50              i++;
51              k++;
52          }
53
54          while (j < n2){
55              arr[k] = rightArr[j];
56              j++;
57              k++;
58          }
59      }
```

```
62  // Merge Sort
63  void mergeSort(int arr[], int left, int right){
64      if (left < right){
65          int middle = left + (right - left) / 2;
66          mergeSort(arr, left, middle);
67          mergeSort(arr, middle + 1, right);
68          merge(arr, left, middle, right);
69      }
70  }
```

```
54      cout << "\nMERGE SORT ALGORITHM" << endl;
55      mergeSort(mergeArray, 0, SIZE - 1);
56      cout << "Sorted Array using Merge Sort:" << endl;
57      displayArray(mergeArray, SIZE);
```

```
MERGE SORT ALGORITHM
Sorted Array using Merge Sort:
1 4 14 40 41 61 63 64 73 76
87 116 128 145 151 168 181 186 197 201
214 235 235 241 264 279 283 303 309 333
345 362 367 369 383 407 427 433 452 479
480 485 489 500 501 515 517 534 542 553
570 596 597 600 613 615 615 621 621 622
623 624 659 662 669 673 703 713 725 729
732 743 752 759 761 766 769 770 773 774
788 811 812 812 821 837 853 855 857 871
884 907 910 911 937 960 961 980 992 994
```

Observations	The Merge Sort successfully sorted the 100 elements in ascending order. The array was repeatedly divided into smaller subarrays, then the sorted subarrays were merged to produce the final sorted array.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```
73  // Partition function for Quick Sort
74  int partition(int arr[], int low, int high){
75      int pivot = arr[high];
76      int i = low - 1;
77      for (int j = low; j < high; j++){
78          if (arr[j] < pivot){
79              i++;
80              int temp = arr[i];
81              arr[i] = arr[j];
82              arr[j] = temp;
83          }
84      }
85
86      int temp = arr[i + 1];
87      arr[i + 1] = arr[high];
88      arr[high] = temp;
89      return i + 1;
90  }
91
92
93  // Quick Sort
94  void quickSort(int arr[], int low, int high){
95      if (low < high){
96          int pivot = partition(arr, low, high);
97          quickSort(arr, low, pivot - 1);
98          quickSort(arr, pivot + 1, high);
99      }
100 }
101
102 #endif
```

```
59      cout << "\nQUICK SORT ALGORITHM" << endl;
60      quickSort(quickArray, 0, SIZE - 1);
61      cout << "Sorted Array using Quick Sort:" << endl;
62      displayArray(quickArray, SIZE);
63
```

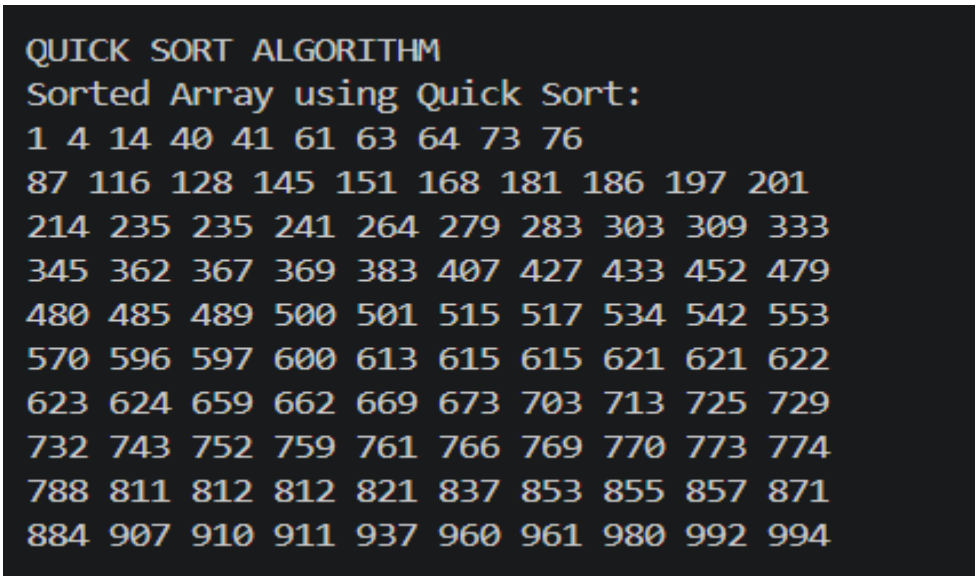
	
Observations	Quick Sort successfully arranged 100 random values in ascending order. The algorithm selected a pivot, partitioned the elements around the pivot, and recursively sorted the left and right subarrays.

Table 8-4. Quick Sort Algorithm

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other
sorting algorithms? Demonstrate an example.

Yes, other sorting methods like Shell Sort, Merge Sort, or Insertion Sort can be a substitute for the left and right sublists that are created during the partition process of Quick Sort. For instance, after the partition process, the left sublist can be sorted using Shell Sort while the right sublist can be sorted using Insertion Sort. Provided that both sublists have been sorted appropriately, they can be merged together with the pivot to create the entire sorted array.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

In such a case, Quick Sort will usually exhibit very good time efficiency as it will divide the array based on a pivot value and sort the subarrays formed in a recursive fashion. On the other hand, Merge Sort will have O(N log N) as it will continually divide the array into smaller and smaller pieces, and merge the array, which will take log N stages with N items being processed at each stage.

CONCLUSION/LEARNINGS/ANALYSIS:

In this activity, I learned how Shell Sort, Merge Sort, and Quick Sort can be used to arrange a dataset in ascending order. I learned that Shell Sort improves the sorting process by using gaps, while Merge Sort uses a divide-and-conquer approach by dividing and merging subarrays. Quick Sort uses a pivot to partition the array into smaller parts before sorting them recursively.

During the procedure, I created an array containing 100 random values and used copies of the same dataset for the three sorting algorithms. The results showed that all three algorithms were able to successfully sort the values. I also learned that different sorting techniques have different ways of organizing and processing data.

From the supplementary activity, I learned that the sublists created by Quick Sort can also be sorted using other sorting techniques. I also understood why Merge Sort and Quick Sort can achieve O(N log N) time complexity when their data is divided into reasonably balanced parts.

Overall, I think I was able to complete the activity successfully and understand the basic implementation of the three sorting algorithms. However, I still need to improve my understanding of recursion, partitioning, merging, and the differences in performance between sorting algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI only paves the way for me to debug my errors in the code, and suggest a better way to implement and format my code for a cleaner output. Other than that, no assistance was called.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.